

Colas Estáticas

con Vectores en Python

Estructuras de Datos

Implementación práctica con arreglos de tamaño fijo

FIFO

Enqueue

Dequeue

Front

Final

F

R

O

N

T

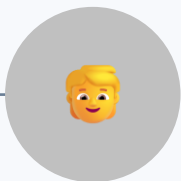
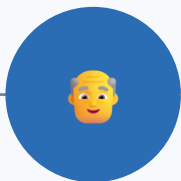
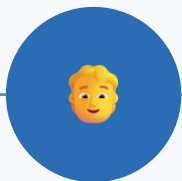
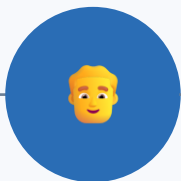


¿Qué es una Cola? (Queue)

El principio FIFO: Primero en entrar, primero en salir



SALE PRIMERO ←



→ ENTRA AL FINAL

FRONT (frente)

FINAL (fondo)



FIFO

First In, First Out
El primero en llegar es el primero en ser atendido



Analogías

- Fila en el banco
- Cola en la impresora
- Buffer de red



Operaciones

- Enqueue → agregar al final
- Dequeue → quitar al frente



Pregunta de Reflexión #1

Piensa antes de responder...

¿En qué situaciones de la vida cotidiana usamos el principio FIFO sin darnos cuenta?

¿Y puedes pensar en algún caso donde NO sería justo usar FIFO?



Urgencias médicas



Abordaje prioritario



Cola de impresión



Matchmaking en juegos



Cola Estática vs Cola Dinámica

¿Cuándo usar cada una?

◆ Cola Estática (Vector)

Memoria ✓ Contigua y predecible

Tamaño ⚠ Fijo desde el inicio

Velocidad ✓ Acceso $O(1)$ muy rápido

Implementación ✓ Simple con lista Python

Riesgo ✗ Overflow si se llena

Uso típico Sistemas embebidos, buffers

◆ Cola Dinámica (Lista enlazada)

⚠ Fragmentada (punteros)

✓ Crece dinámicamente

✓ Inserción $O(1)$ también

⚠ Más compleja (nodos)

✗ Overhead de memoria

Colas de tamaño variable



Representación en Memoria

Cómo se ve una cola estática internamente

Un vector de tamaño fijo con dos índices: front y final



MAX_SIZE
6
Capacidad total del vector

front
0
Índice del primer elemento

final
2
Índice del último elemento

size
3
Elementos en la cola ahora



Implementación Base en Python

Definición de la clase ColaEstatica

```
class ColaEstatica:
    def __init__(self, max_size):
        self.max_size = max_size      # Capacidad máxima
        self.datos = [None] * max_size # Vector/lista fija
        self.frente = -1              # Índice del frente
        self.final = -1               # Índice del final
        self.size = 0                 # Contador de elementos
```

Capacidad máxima definida
al crear la cola

Lista Python simula
un arreglo estático

front siempre apunta
al primer elemento

final = -1 indica
cola vacía al inicio

size cuenta cuántos
elementos hay

+ Operación: Enqueue (Encolar)

Agregar un elemento al final de la cola



```
def enqueue(self, valor):  
    if self.is_full():  
        raise Exception("Cola llena")  
    self.final = (self.final + 1) % self.max_size  
    self.datos[self.final] = valor  
    self.size += 1  
    return True
```



¿Por qué el módulo %?

El operador % (módulo) hace que el índice vuelva al inicio cuando llega al final.

Esto crea una cola CIRCULAR y evita desperdicio de espacio.

Ejemplo:

$(5 + 1) \% 6 = 0$ ← vuelve al inicio

1

Verificar si está llena

Si `size == max_size` → error

2

Avanzar final

`final = (final + 1) % max_size`

3

Guardar el dato

`datos[final] = valor`

4

Incrementar size

`size += 1`

Operación: Dequeue (Desencolar)

Quitar y retornar el elemento del frente



```
def dequeue(self):  
    if self.is_empty():  
        raise Exception("Cola vacía")  
    valor = self.datos[self.front]  
    self.datos[self.front] = None  
    self.front = (self.front + 1) % self.max_size  
    self.size -= 1  
    return valor
```



Diferencia clave

Enqueue agrega al FINAL
Dequeue quita del FRONT

Nunca debes mover todos
los elementos hacia adelante.

Solo se mueven los índices
front y final — mucho más
eficiente: $O(1)$ siempre.

1

Verificar si está vacía

Si `size == 0` → error

3

Limpiar la celda

`datos[front] = None`

5

Decrementar size

`size -= 1`

2

Guardar valor del frente

`valor = datos[front]`

4

Avanzar front

`front = (front + 1) % max_size`

6

Retornar el valor

`return valor`



Operaciones Auxiliares

isEmpty, isFull, peek, display

is_empty()

```
def is_empty(self):  
    return self.size == 0
```

Retorna True si no hay elementos en la cola.
Siempre verificar antes de hacer Dequeue.

is_full()

```
def is_full(self):  
    return self.size == self.max_size
```

Retorna True si la cola alcanzó su límite.
Siempre verificar antes de hacer Enqueue.

peek()

```
def peek(self):  
    if self.is_empty():  
        return None  
    return self.datos[self.front]
```

VER el frente sin quitarlo.
Útil para consultar el próximo en salir.

display()

```
def display(self):  
    print(f"Cola [{self.size}/{self.max_size}]")  
    for i in range(self.size):  
        idx=(self.front+i)%self.max_size  
        print(self.datos[idx])
```

Muestra el contenido ordenado de la cola, desde front hasta final, respetando la circularidad.



Pregunta de Reflexión #2

Análisis de complejidad y diseño

¿Por qué las operaciones enqueue y dequeue tienen complejidad $O(1)$ y no $O(n)$?



Sin mover datos

Solo movemos índices
no los elementos en sí



Tiempo constante

Siempre la misma cantidad
de operaciones simples



Índices aritméticos

front y final calculados
con una sola operación



Acceso directo

datos[front] accede
directo a la posición

vs. Si moviéramos todos los elementos: $O(n)$ — mucho más lento



Cola Circular con Módulo

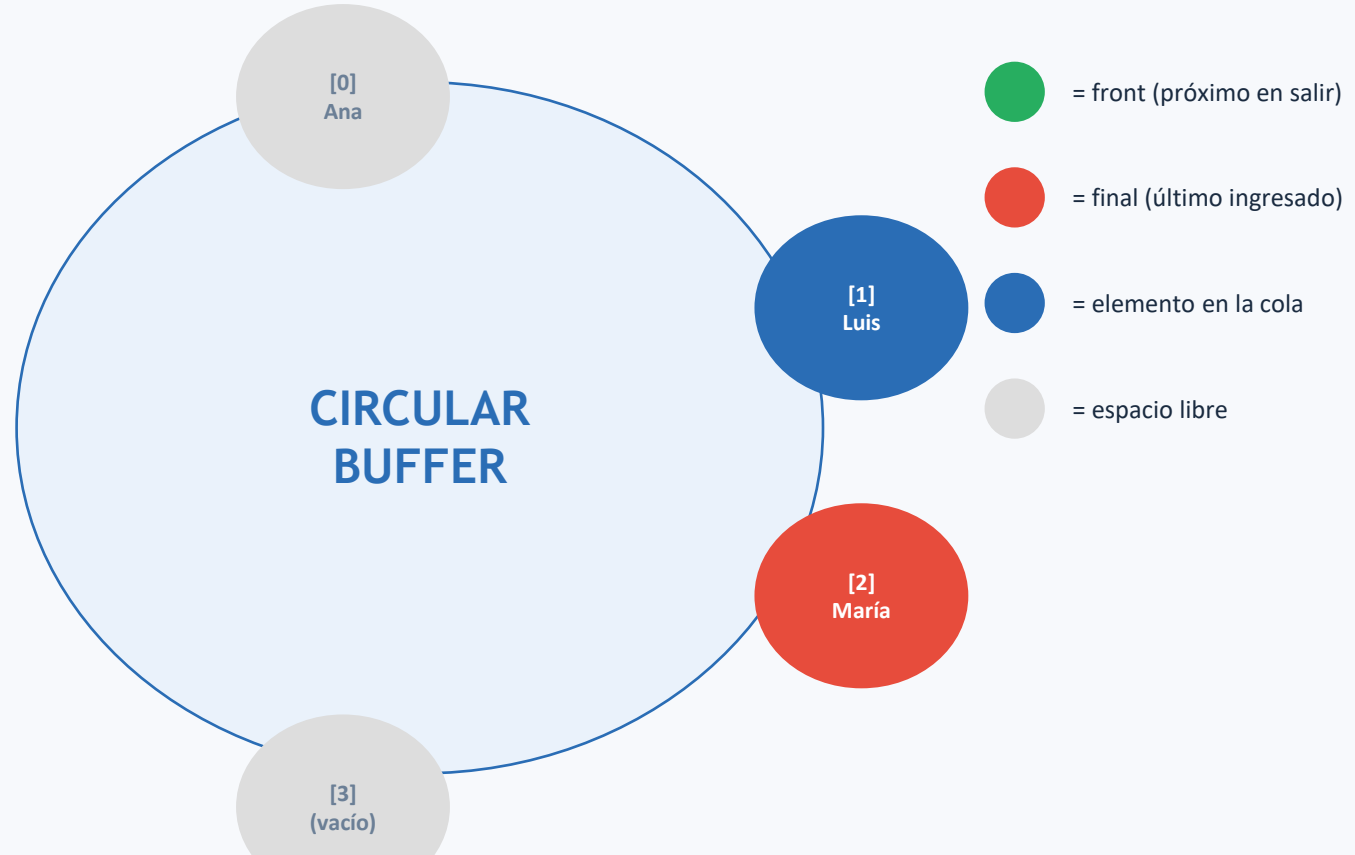
Reutilizar el espacio liberado por Dequeue

¿Por qué circular?

Sin circularidad:
Al hacer dequeue
el front sube y el
espacio liberado
se pierde.

Con módulo %:
El final puede
volver al índice 0
y usar el espacio
liberado.

Fórmula:
$$\text{nuevo} = (\text{índice} + 1) \% \text{MAX}$$





Ejemplo Completo de Uso

Simulación paso a paso de la cola



```
# Crear cola con capacidad 3
cola = ColaEstatica(3)

# Encolar tres elementos
cola.enqueue("Ana")    # front=0, final=0
cola.enqueue("Luis")   # front=0, final =1
cola.enqueue("María")  # front=0, final =2

# Observar frente
print(cola.peek())     # → 'Ana'

# Desencolar
print(cola.dequeue())  # → 'Ana'
print(cola.dequeue())  # → 'Luis'
```

Estado de la Cola

Operación	front	final	size
inicio	-	-1	0
enqueue('Ana')	0	0	1
enqueue('Luis')	0	1	2
enqueue('María')	0	2	3
peek()	0	2	3
dequeue()	1	2	2
dequeue()	2	2	1



Manejo de Errores y Casos Borde

Overflow, Underflow y uso seguro



Overflow

Cola llena — enqueue fallido

```
cola = ColaEstatica(2)
cola.enqueue("A")
cola.enqueue("B")
# Esta línea causa error:
cola.enqueue("C") # OverflowError
```



Solución recomendada:

Siempre verificar
`is_full()` antes
de encolar.

O usar `try/except`
para capturar
el error.



Underflow

Cola vacía — dequeue fallido

```
cola = ColaEstatica(3)
# Sin encolar nada:
cola.dequeue() # ← Error!

# Forma segura:
if not cola.is_empty():
    dato = cola.dequeue()
```



Solución recomendada:

Siempre verificar
`is_empty()` antes
de desencolar.

Nunca asumir
que hay datos
en la cola.



Pregunta de Reflexión #3

Diseño y aplicaciones reales

Imagina que eres desarrollador/a de un sistema de atención a clientes en un banco.
¿Cómo implementarías la cola de espera?



¿Qué tamaño darías a la cola estática?

¿Cuántos clientes pueden esperar máximo?



¿Qué pasa si la cola se llena?

¿Error, cola de espera virtual, o turnos?



¿Necesitas prioridades especiales?

Adultos mayores, discapacitados, VIP...



¿Cola estática o dinámica?

¿El aforo del banco es variable o fijo?



Aplicaciones Reales de Colas

Dónde se usan en sistemas del mundo real



Cola de Impresión

El sistema operativo mantiene una cola de trabajos. El primero en enviarse, el primero en imprimirse.



Buffer de Red

Los paquetes de datos se colocan en cola mientras esperan ser transmitidos por la red.



Playlist de Música

Las canciones se agregan al final y se reproducen desde el frente de la lista.



Scheduler de CPU

El sistema operativo gestiona los procesos en espera con colas de prioridad.



Sistemas de Tickets

Tiendas, bancos, hospitales: numeritos de atención implementan FIFO explícitamente.



Cola de Mensajes

Servidores de mensajería como RabbitMQ usan colas para comunicar microservicios.



Ejercicios Prácticos

Pon en práctica lo aprendido

1

Completar la Clase

Nivel: Básico

Implementa los métodos faltantes:

- `size()` → retorna número de elementos
- `clear()` → vacía la cola completamente
- `__str__()` → representación como string

2

Simulador de Banco

Nivel: Intermedio

Crea una cola de 5 personas.

Encola: 'Pedro', 'Sofía', 'Javier'

Desencola 1, Encola 'Valentina'

Muestra el estado final con `display()`

3

Inversión de Cola

Nivel: Avanzado

Implementa una función que invierta los elementos de una cola estática
Usando solo operaciones de cola
(Pista: usa una pila auxiliar)

4

Cola de Prioridad Básica

Nivel: Extra

Modifica la clase para agregar un campo 'prioridad' por elemento.
Encola según prioridad (1=alta, 3=baja)
¿Cómo cambia la implementación?



Preguntas de Reflexión Final

Cierre de clase

1

¿Cuándo elegirías una cola estática sobre una dinámica, y por qué?

2

¿Qué ventaja da el módulo % al implementar la cola circular frente a la cola lineal?

3

Si debieras implementar una cola de emergencias médicas, ¿cambiarías el principio FIFO? ¿Cómo?

4

¿Qué estructura de datos usarías si necesitaras acceder al elemento del centro? ¿Sigue siendo una cola?

 *Recuerda: No hay respuestas únicas — el diseño de estructuras depende siempre del contexto y los requisitos del sistema.*



Mapa de Conceptos

Todo lo que aprendiste hoy

